



VIT[®]
BHOPAL

VisionLens

*A Computer Vision Based Campus Lost-and-Found
Visual Matching System*

PROJECT REPORT

COMPUTER VISION (CSE3010)

Slot F11+F12

SUBMITTED BY:

Student Name: Shashank Kumar
Registration No: 24BAI10032
Program: B.Tech Computer Science
& Engineering

SUBMITTED TO:

Faculty Name: Dr. Siddharth Singh
Chouhan
Designation: Associate Professor
School: SCAI, VIT Bhopal University

1. Introduction

VisionLens is a project that made me computer vision based image retrieval system which is designed for finding lost items on campus. The system takes a photo of a lost item and search a collection of found item photos for visually similar images.

The project uses a pretrained ResNet-18 convolutional neural network as a feature extractor instead of using a training for classification model from scratch. These vectors are normalized and compared using cosine similarity. The system ranks the gallery images and returns the very certain Top-5 visually similar results together with available item labels and similarity scores.

The implementation was intentionally kept small and modular so that the core computer vision workflow could be completed, tested, documented, and evaluated within the project timeframe.

2. Problem Statement

In a campus environment, lost-and-found items may be represented by a large collection of photos. Manually checking all photos one by one can be very time-consuming and may make it difficult to identify visually similar items quickly too.

VisionLens makes sure to tackle this problem by providing image-based visual retrieval. Instead of requiring the user to know an exact item category or filename, the system compares the visual representation of a query image against a searchable gallery of found-item images.

3. Objectives

- Develop a computer vision based image retrieval system for lost-and-found item photographs.
- Load image data and multi-label annotations from the supplied dataset.
- Preprocess query and gallery images using the transformations expected by the pretrained model.
- Extract meaningful visual features using a pretrained ResNet-18 network.
- Represent each image as a 512-dimensional feature vector.
- Calculate visual similarity using cosine similarity.
- Rank gallery images and return the Top-5 visually similar results.
- Cache extracted gallery features so repeated searches do not require recomputing all gallery embeddings.
- Provide automated tests for important project modules.

4. Scope

The current scope is deliberately focused on the core retrieval pipeline.

In Scope

- Image dataset loading and CSV label parsing.
- Image preprocessing.
- Pretrained CNN feature extraction.
- Cosine similarity based retrieval.
- Top-K ranking, with Top-5 used in the demonstration.
- Cached gallery feature storage.

- Command-line demonstration.
- Automated unit/integration-style tests for core modules.
- Technical documentation and design artefacts.

Out of Scope

- Web interface.
- User authentication.
- Mobile application.
- Live camera processing.
- Cloud deployment.
- Training a CNN from scratch.
- Complex administrative dashboard.
- Large-scale production database.

5. Functional Requirements

Requirement	My Project
Load dataset images and annotations.	Image records with paths and labels.
Preprocess a query image.	Model-ready RGB tensor.
Extract image features.	Normalized 512-D feature vector.
Build/load the searchable gallery.	Gallery containing image metadata and features.
Calculate visual similarity.	Cosine similarity score.
Rank gallery images.	Results ordered by descending similarity.
Return Top-5 matches.	Five highest-scoring gallery results with labels and scores.

6. Non-Functional Requirements

Requirement	My Project
Usability	Simple command-line workflow and readable result output.
Performance	Gallery embeddings are cached in features/gallery.pt.
Maintainability	Separate modules for loading, extraction, similarity, and search.
Reliability	Missing files and individual extraction errors are handled explicitly.
Resource Efficiency	CPU execution is used for broad laptop compatibility; features are reused.
Testability	Automated pytest tests cover core functionality.

7. Dataset Description

The project uses the supplied Lost-Found-Items.v1i.multiclass dataset. The dataset is organized into train, valid, and test splits, with an accompanying `_classes.csv` file in each split. The CSV annotations support multiple active labels for an image.

Split	Images	Role in VisionLens
Train	1,959	Searchable gallery used to build cached visual features.
Valid	183	Validation/tuning split available for checking the approach.
Test	95	Query/evaluation split used for demonstrations and testing.

The dataset contains item categories including examples such as backpacks, cards, glasses, keys, wallets, watches, bags, and electronic devices. The system does not train a 51-class classifier; instead, it uses the images as a visual retrieval gallery.

A limitation of the supplied dataset is that some filenames and image groups indicate augmentation or near-duplicate variants. Therefore, retrieval demonstrations should not be interpreted as independent statistical evidence without additional duplicate-aware evaluation.

8. System Architecture

VisionLens follows a modular pipeline. DatasetLoader reads image filenames and multi-label annotations. FeatureExtractor performs preprocessing and obtains ResNet-18 embeddings. SimilarityEngine calculates cosine similarity between normalized feature vectors. SearchEngine coordinates gallery construction/loading, query processing, ranking, and Top-K retrieval.

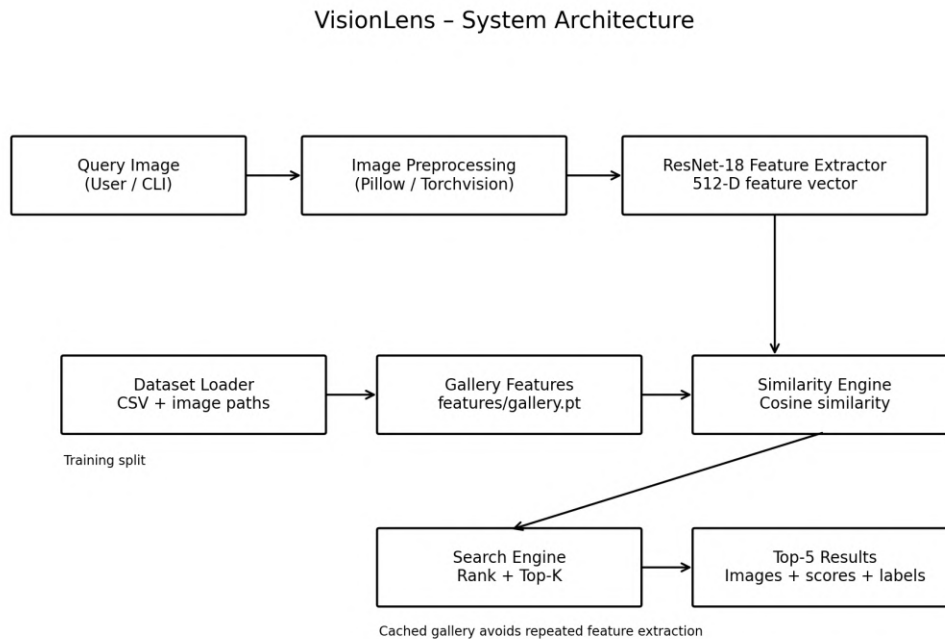


Figure 1. VisionLens system architecture.

9. Workflow

The workflow begins with a query image. The image is loaded and transformed using the preprocessing associated with the pretrained ResNet-18 weights. A 512-dimensional feature vector is extracted. The vector is compared against the cached training gallery. Results are sorted by similarity score and the five highest-scoring images are displayed.

VisionLens – Workflow

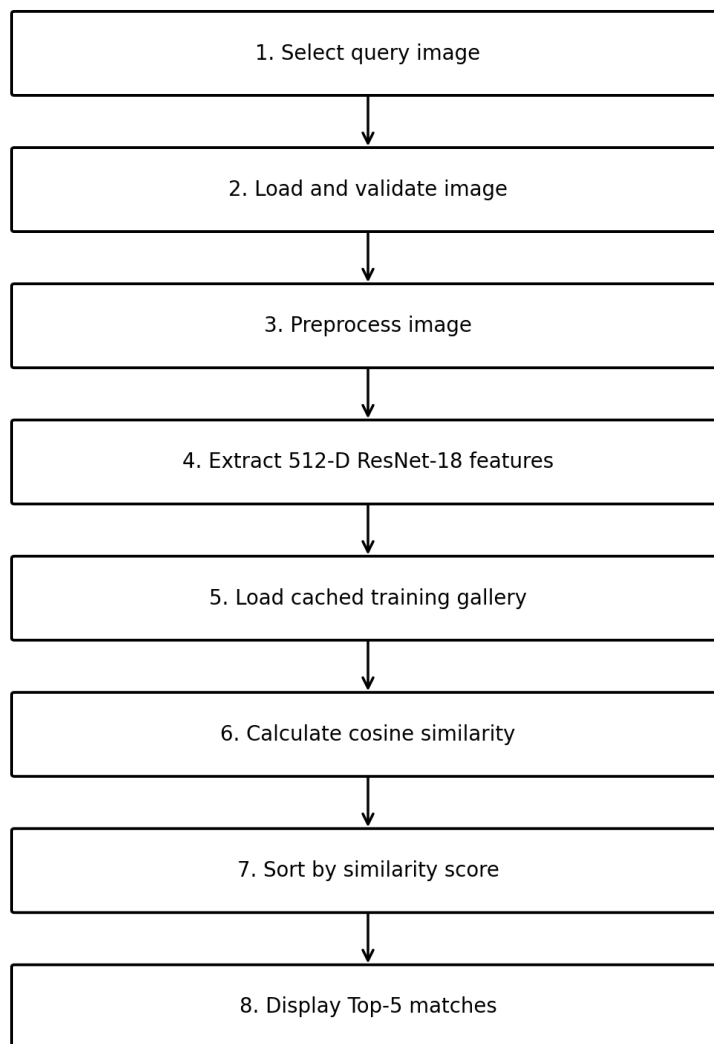


Figure 2. VisionLens workflow.

10. UML Diagrams

10.1 Use Case Diagram

The main actor is the user who provides a query image and views the retrieved results. The system performs visual search, calculates similarity, and exposes the Top-5 results, scores, and labels.

VisionLens - Use Case Diagram

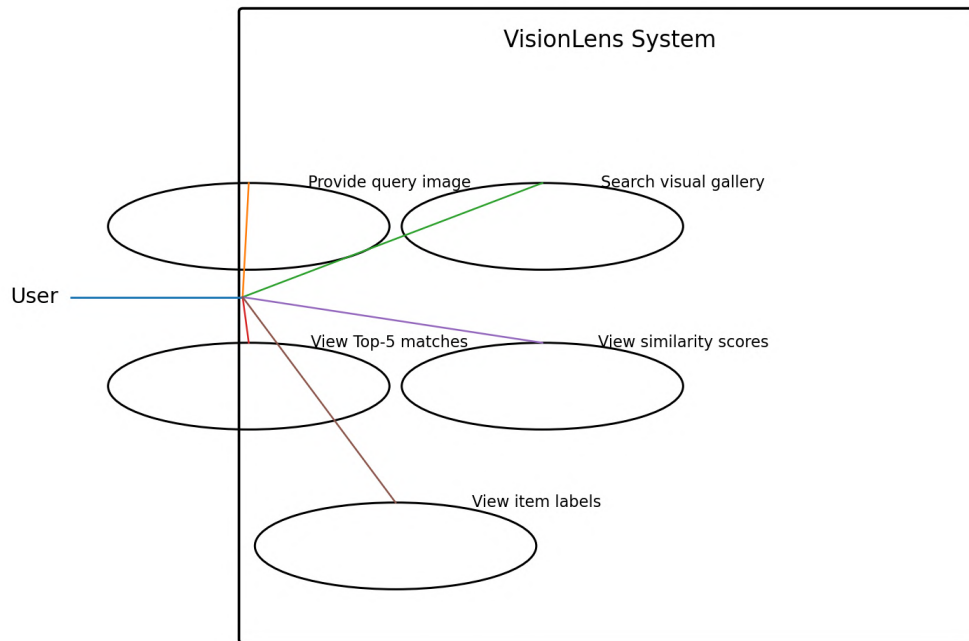


Figure 3. VisionLens use case diagram.

10.2 Class / Component Diagram

The component view maps directly to the implemented Python modules: DatasetLoader, FeatureExtractor, SimilarityEngine, SearchEngine, and the pytest test suite.

VisionLens – Class / Component Diagram

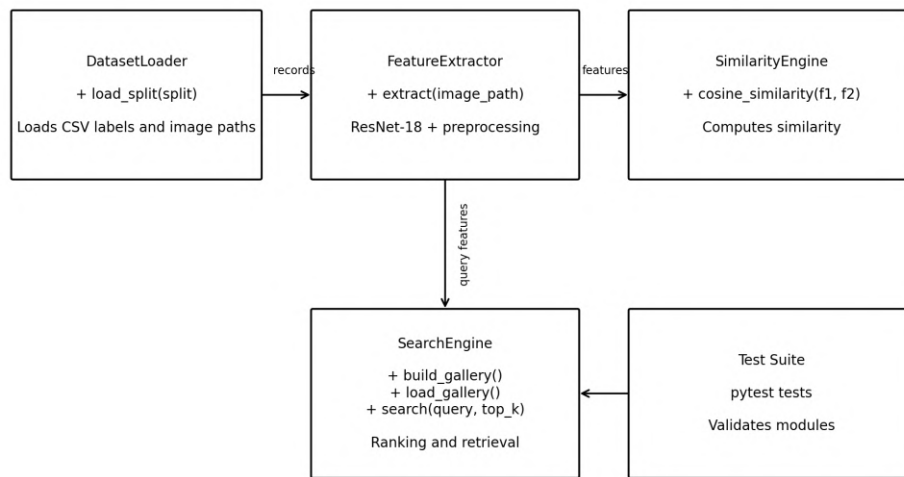


Figure 4. VisionLens class/component diagram.

10.3 Sequence Diagram

The sequence diagram shows the interaction from query submission through feature extraction, gallery retrieval, similarity calculation, and result display.

VisionLens – Sequence Diagram

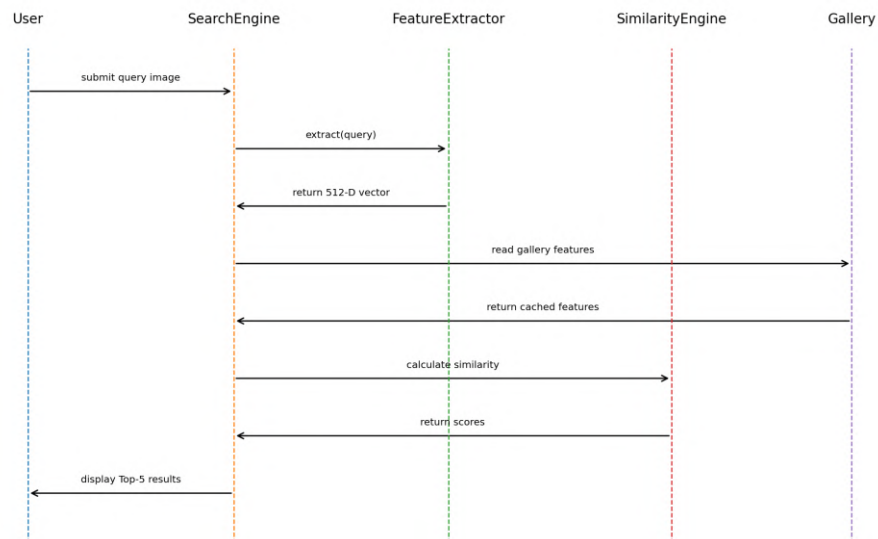


Figure 5. VisionLens sequence diagram.

11. Computer Vision / ResNet-18 Approach

VisionLens uses ResNet-18 from Torchvision with pretrained weights. The final classification layer is removed so that the remaining network acts as a visual feature extractor.

1. Load the input image and convert it to RGB.
2. Apply the preprocessing associated with the pretrained ResNet-18 weights.
3. Pass the image through the convolutional network.
4. Remove the final classification layer and obtain a 512-dimensional representation.
5. Flatten the representation and apply L2 normalization.
6. Compare the normalized query representation with normalized gallery representations.

Because the feature vectors are L2-normalized, their dot product is mathematically equivalent to cosine similarity. The similarity implementation therefore uses the normalized vector dot product.

12. Implementation

The implementation is divided into focused modules:

Module	Responsibility
cv/dataset_loader.py	Loads train/valid/test CSV annotations and creates image records.
cv/feature_extractor.py	Loads pretrained ResNet-18, preprocesses images, and extracts 512-D features.
cv/similarity.py	Computes cosine similarity between normalized feature vectors.
cv/search_engine.py	Builds/loads the gallery, performs query search, ranks results, and returns Top-K.
tests/test_visionlens.py	Automated tests for loading, feature extraction, and similarity behavior.
features/gallery.pt	Cached gallery features and metadata generated by the search engine.

The implementation uses Python, PyTorch, Torchvision, Pillow, OpenCV, NumPy, scikit-learn, and pytest. The current project intentionally does not include a web application.

13. Testing — 4/4 Passed

Automated tests were executed from the project root using pytest. The final test run collected four tests and all four passed.

Test command:

python -m pytest -v

```

1 import torch
2
3 from cv.dataset_loader import DatasetLoader
4 from cv.feature_extractor import FeatureExtractor
5 from cv.similarity import SimilarityEngine
6
7
8
9
10 class SearchEngine:
11

```

```

(.venv) PS I:\Learning on Building a Model\VisionLens> .\.venv\Scripts\python.exe -m pytest -v
===== test session starts =====
platform win32 -- Python 3.11.9, pytest-9.1.1, pluggy-1.6.0 -- I:\Learning on Building a Model\VisionLens\.venv\Script
s\python.exe
cachedir: .pytest_cache
rootdir: I:\Learning on Building a Model\VisionLens
collected 4 items

tests/test_visionlens.py::test_dataset_loader PASSED [ 25%]
tests/test_visionlens.py::test_feature_extraction PASSED [ 50%]
tests/test_visionlens.py::test_cosine_similarity PASSED [ 75%]
tests/test_visionlens.py::test_different_images_produce_similarity_score PASSED [100%]

===== 4 passed in 7.24s =====
(.venv) PS I:\Learning on Building a Model\VisionLens>

```

Test	Purpose	Result
test_dataset_loader	Checks that the test split loads records, labels, and existing image paths.	PASSED
test_feature_extraction	Checks that feature extraction produces a 512-dimensional vector.	PASSED
test_cosine_similarity	Checks that comparing an image with itself produces a similarity above 0.99.	PASSED
test_different_images_produce_similarity_score	Checks that similarity remains within the valid cosine range.	PASSED

Final execution result: 4 passed in 7.24 seconds.

14. Results

The gallery-building stage successfully processed all 1,959 training images and stored their features in features/gallery.pt. A second execution loaded the cached gallery rather than recomputing the 1,959 embeddings.

Example retrieval result:

The screenshot shows a VS Code editor with a file named `search_engine.py` open. The file contains a function that searches for test images in a dataset. The terminal output shows the execution of the script, which processed 1900/1959 images and returned the top 5 matches, all labeled as 'Backpack'.

```

97         (dataset_root / "test").glob("*.jpg")
98     )
99
100     if not test_images:
101         raise RuntimeError(
102             "No test images found."

```

```

Processed 1900/1959 images

(.venv) PS I:\Learning on Building a Model\VisionLens> .\.venv\Scripts\python.
exe cv/search_engine.py
20240315_131735_jpg.rf.92e69a24f02879586d9b65591d1669ae.jpg

Top 5 matches:
-----
1. IMG_20240316_090402_109-0-_jpg.rf.49853ff4fae9ef2bd5b4c3d4eeec6d00.jpg | Sc
ore: 0.7797 | Labels: ['Backpack']
2. IMG_20240323_154546_jpg.rf.045f93cbec7fef0f04942f336f4f6b05.jpg | Score: 0.
7782 | Labels: ['Backpack']
3. IMG_20240316_172819_jpg.rf.a2c4a2e630f2634cd688279289e9996e.jpg | Score: 0.
7711 | Labels: ['Backpack']
4. IMG_20240323_154546_jpg.rf.b93d290987a96d7b6333b93cd118d67c.jpg | Score: 0.
7665 | Labels: ['Backpack']
5. IMG_20240323_154546_jpg.rf.27dbbe3d4a4270f4a77f1afa60703ade.jpg | Score: 0.
7650 | Labels: ['Backpack']
(.venv) PS I:\Learning on Building a Model\VisionLens>

```

Rank	Retrieved Item	Similarity	Label
1	IMG_20240316_090402_109-0-...	0.7797	Backpack
2	IMG_20240323_154546_...	0.7782	Backpack
3	IMG_20240316_172819_...	0.7711	Backpack
4	IMG_20240323_154546_...	0.7665	Backpack
5	IMG_20240323_154546_...	0.7650	Backpack

This example demonstrates that the retrieval pipeline can identify visually similar images from the gallery and return ranked results with similarity scores and labels. The example is a qualitative sanity check rather than a formal accuracy measurement.

15. Design Decisions & Rationale

Decision	Rationale/ Why I used them
Use pretrained ResNet-18	I used a pretrained model because training a CNN from scratch was outside the time and data available for this project
Use feature retrieval instead of classification	The requirement was to find visually similar photographs, not simply predict one class.
Use cosine similarity	It gives a simple way to compare the normalized feature vectors produced by the model.
Use the train split as the gallery	The training split provides the larger image collection, while the validation and test images can

	remain separate.
Cache gallery features	Generating features for all 1,959 gallery images repeatedly would waste processing time
CPU execution	I wanted the project to run on a normal coursework laptop without depending on a GPU.
Modular Python design	Separating the components made it easier to test and debug individual parts.
CLI instead of web UI	I kept the interface simple so that most of the project effort remained on the computer-vision pipeline.

16. Challenges & Learnings

- Dataset paths must also match the actual project structure; the final dataset location is data/Lost-Found-Items.v1i.multiclass.
- The supplied CSV files contain multi-label annotations, so the loader was designed to preserve all active labels for an image.
- Extracting features for all 1,959 gallery images took noticeably more processing than extracting features for a single query. Because of this, I added gallery feature caching instead of repeating the complete extraction process every time the program was run.
- A visually similar result is not necessarily an exact identity match; retrieval quality depends on image appearance, viewpoint, lighting, and background.
- Automated tests provides a repeatable way to verify important components before submission.
- A project can satisfy core computer vision objective without adding a large web stack that is outside the essential scope.

17. Limitations of the Project

- The current system measures visual similarity rather than confirming that two images show the exact same physical object.
- The quality of retrieval can be affected by lighting, viewpoint, scale, occlusion, background, and image quality.
- The implementation currently uses a simple exhaustive comparison against the cached gallery.
- The project does not provide a web interface or authentication layer.
- The supplied dataset includes augmentation/near-duplicate patterns, so independent retrieval evaluation should be designed carefully.
- The current automated tests validate core behavior but do not constitute a large-scale retrieval benchmark.

18. Future Enhancements

- Add a lightweight web interface for image upload and visual result display.
- Add category filtering using the existing labels.
- Add complementary visual cues such as dominant color.
- Use a larger and more diverse lost-and-found dataset.
- Introduce duplicate-aware and category-aware retrieval evaluation metrics such as Recall@K.

- Investigate more advanced image embedding models if additional project time and resources are available.
- Add an indexed vector search method for much larger galleries.

19. References

7. VITyarthi. Build Your Own Project – General Project Instructions & Submission Guidelines. Course-provided project guideline.
8. He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).
9. PyTorch Documentation. PyTorch: An Open Source Machine Learning Framework. <https://pytorch.org/docs/>
10. Torchvision Documentation. Models and pretrained computer vision models. <https://docs.pytorch.org/vision/>
11. Roboflow dataset export / annotation format documentation, as applicable to the supplied Lost-Found-Items dataset.

Project Structure

```
VisionLens/
├── app/
├── cv/
│   ├── dataset_loader.py
│   ├── feature_extractor.py
│   ├── search_engine.py
│   └── similarity.py
├── data/
│   └── Lost-Found-Items.v1i.multiclass/
├── docs/
│   └── diagrams/
├── features/
│   └── gallery.pt
├── tests/
│   └── test_visionlens.py
├── README.md
├── statement.md
└── requirements.txt
```